

2026-05-06-p1-f20-theme-system-design

Phase 1 / Feature 20 — Theme System (FINAL Phase 1 feature)

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

Note (programme position): F20 is the **last** Phase 1 feature. T09 (close-out) records “Phase 1 of CLI-Agent Fusion programme complete” in PROGRESS.md.

1. Goal

Ship a real, end-to-end **5-role semantic theme system** for the HelixCode CLI agent so that LLM token streams, tool result frames, and slash-command output can be colourised with a *consistent, role-based palette* that adapts to the active terminal’s color depth and to the user’s preference (built-in `dark / light / none`, or a user YAML override). When the F18 renderer is in plain mode (non-TTY destinations: pipes, CI logs, `>file`, dumb terminals), zero color bytes **MUST** be emitted regardless of the active theme — the no-leak invariant is structurally enforced.

Three concrete user surfaces ship together:

1. **theme package** (`helix_code/internal/theme/`) — five semantic roles (`Info / Warn / Error / Highlight / Dim`; Q1=A) backed by per-depth `Color` variants (`ANSI16 / ANSI256 / Truecolor`). Three built-in themes (`dark / light / none`; Q2=C) that always ship with the binary and never require disk I/O. An optional user YAML override at `$XDG_CONFIG_HOME/helixcode/theme.yaml` (Q2=C) that can replace any subset of the built-in palette without forcing the user to re-spell every role. A `Styler` that wraps text with role-coded ANSI escape codes for fancy mode and acts as a no-op pass-through for plain mode.
2. **Renderer integration** (Q1=A consequence) — F18’s `Renderer` is extended through a thin `Stylize(role Role, text string)` string helper invoked at the call sites that already pass text into `WriteToken / RenderTextBlock / RenderLines`. The `Styler` is constructed once at process startup alongside the renderer (same factory pattern as F18 + F19) and threaded through the existing wire points (LLM streaming hook in `handleGenerate`, tool result frame helpers, slash-command output). Plain-mode renderers receive a `Styler` whose `Stylize` is the identity function — no ANSI escape codes leave the package.
3. **Env var + /theme slash command** (Q5=B) — `HELIXCODE_THEME=dark|light|none|<custom-name>` (Q3=C precedence: `env > $COLORFGBG parse > default dark`). Color depth auto-detected (Q4=D) from `$COLORTERM` (`truecolor/24bit → Truecolor`), then `$TERM` (`*-256color → ANSI256`, otherwise `→ ANSI16`; `dumb/unset → Off`). A read-only `/theme` slash command with three subcommands: `/theme status` (active name + depth + source), `/theme list` (all available names: built-in + YAML), `/theme show <name>` (preview the 5 roles in a numbered table with sample text). NO cobra subcommand. Switching the active theme at runtime is OUT OF SCOPE for v1 (deferred to F20.5; see §8).

The plain-mode-zero-color invariant ($Q1=A + Q4=D$ consequence) is **structurally enforced**: the Styler's `Stylize` method consults the active `ColorDepth` at call time; `depthOff` (returned for plain renderers, dumb terminals, and explicit none theme) makes `Stylize(role, text) == text` byte-for-byte. A unit test scans the captured plain-mode writer for any byte in `{0x1b}` (ESC) and FAILS on any hit. The Challenge harness includes a dedicated PLAIN-MODE-ZERO-COLOR phase that constructs a real `plainRenderer` + a real Styler with a colourful theme + writes 100 styled tokens through the production code path and asserts the captured bytes contain ZERO `0x1b` characters.

The dark/light/none built-ins are concrete and load-bearing — see §3.4 for the exact ANSI codes per role × depth. `none` is the explicit “no color, even on a TTY” theme (distinct from the `depthOff` plain-mode path) — its codes are empty strings on every depth, so `Stylize(role, text) == text` regardless of renderer mode.

Out of scope for v1: bold / italic / underline / blink (defer to F20.5 — Constitutional anchors are unchanged but the theme YAML schema gains optional `bold/italic/underline` fields); background colors (defer to F20.5); per-tool-type custom colors (e.g. “all `lsp_diagnostic` results in red regardless of role”); gradient text; transparent / faded text; mid-process theme reload on YAML file change (the YAML is read once at startup); GUI theme synchronisation (the desktop Fyne UI under `applications/desktop/` reads `assets/colors/helix-theme.css` directly — that path is left intact and not coupled to the CLI theme); 256-color truecolor approximation algorithms beyond a literal lookup (an ANSI256 theme uses ANSI256 codes verbatim; a Truecolor theme uses Truecolor codes verbatim; cross-depth fallback within one theme is documented in §5.5).

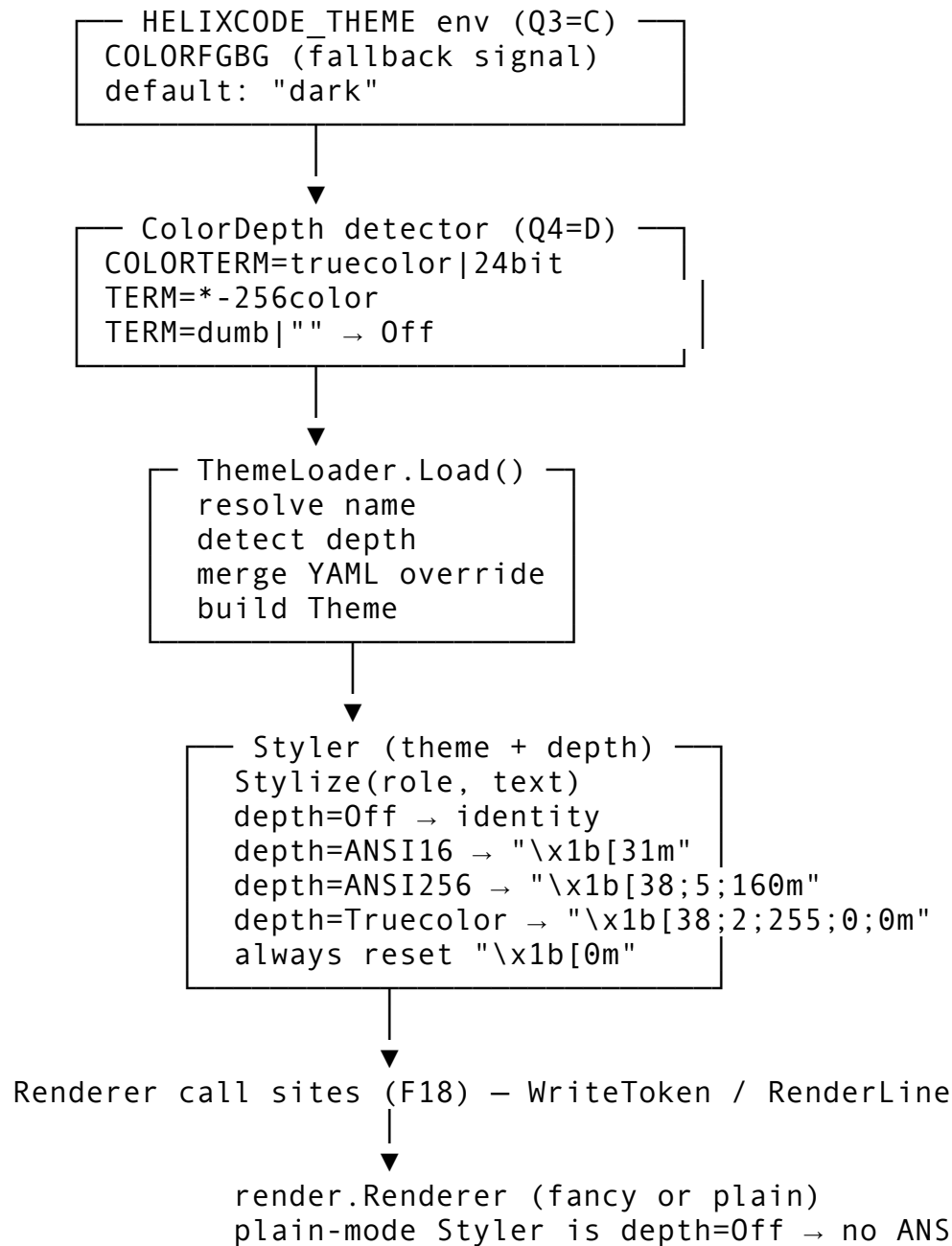
Anti-bluff hot zone (loud): a theme “applied” but no ANSI bytes ever observable in fancy-mode output (the package compiles, the slash command lists the theme, but the call sites still emit raw text); truecolor codes (`\x1b[38;2;<r>;<g>;<b>m`) emitted on a 16-color terminal where they render as visual garbage; plain-mode somehow leaking ANSI through the theme path (because the call site forgot to use the Styler and reached for `Stylize` directly on the package); the env var `HELIXCODE_THEME` honoured but `$COLORFGBG/$COLORTERM/$TERM` detection branches never exercised in tests so a regression silently downgrades every user. Each of these maps to a unit + integration + Challenge phase per §5.2.

2. Architecture

Four layers, all under `helix_code/internal/theme/`, plus thin wiring at the F18 renderer boundary and one slash command:

- Theme **struct** (`types.go`) — the value type carrying the 5-role palette per `ColorDepth`. Concretely, a Theme holds three `[5]Color` arrays (one per depth tier) so a single Theme can be rendered at the depth the terminal actually supports, without a runtime cross-depth conversion (which would bias the palette).
- ThemeRegistry (`builtin.go`) — exposes `BuiltIn()` `map[ThemeName]Theme` returning the three frozen built-ins (`ThemeDark`, `ThemeLight`, `ThemeNone`). Built-ins live in code (no disk read), so a binary with `NO theme.yaml` on disk still has all three available. The registry is read-only after `Build()` — concurrent reads are safe; writes happen only via the loader.
- ThemeLoader (`loader.go`) — resolves the active `ThemeName` per the §3.6 precedence ladder, then merges any matching YAML override on top of the built-in. The loader is the *only* place env vars and disk I/O happen; the rest of the codebase only sees a constructed Styler.

- **Styler(loader.go)** — the runtime façade. Owns one Theme + one ColorDepth. Stylize(role Role, text string) string looks up the theme's Color for role at the current depth and wraps text with the open / reset escape sequences. When ColorDepth == Off, returns text unchanged (identity).



Wire points (existing code; one new line each):

- **Renderer factory boundary:** `cmd/cli/main.go` constructs the renderer at startup (F18 wiring); F20 adds an adjacent `theme.NewStyler(theme.LoaderOptions{...})` call. The Styler is held on the `*CLI` struct (sibling field of `c.renderer`) and threaded into the spots that today pass plain text to F18's helpers. Specifically:
 - `handleGenerate` non-stream path (the `RenderTextBlock(c.renderer, "", text)` call added in F18-T08) becomes `RenderTextBlock(c.renderer, "",`

- `c.styler.Stylize(theme.RoleInfo, text)` for the response body. Errors emitted on the same path are styled with `theme.RoleError`.
- Slash-command output paths (`/lsp`, `/sandbox`, `/edit`, `/sessions`, `/skills`, `/commands`, `/telemetry`, `/subagents`, `/tasks`, `/plan`, `/mcp`, `/permissions`, `/hooks`) — F20 does NOT refactor every one. F20 wires the Styler into the *new* `/theme` command's own output (so the demo of the styling system is itself styled) and wires it into the F18 `RenderTextBlock` helpers in `cmd/cli/main.go::handleGenerate`. Other call sites are migration candidates for F20.5.
- Tool result frame rendering (F18 `RenderLines` glue) — F20 wires the Styler at the call site that classifies a frame as `error` vs `info` (e.g. an `lsp_diagnostic` with `severity=error` styles the heading line via `RoleError`). Concretely: `cmd/cli/main.go` formats the tool-result lines and calls `c.styler.Stylize(role, line)` per line BEFORE passing to `RenderLines`.
- **Slash-command registration** (`cmd/cli/main.go`): one new line under the existing `commands.Registry.Register(...)` block — `c.commandRegistry.Register(commands.NewThemeCommand(c.styler, themeLoader))`. The command name is `theme`; subcommands `status` `/list` `/show` `<name>`.
- **Renderer factory itself**: F18's `render.NewRenderer` is unchanged. The Styler does NOT live inside `internal/render/` — that package is plumbing-only (byte emission). Theme is a higher-level concern that decorates the *content* before it reaches the renderer. This separation keeps the F18 plain-renderer's `bytes.IndexAny(out, "\x1b")` invariant trivially provable: nothing inside `internal/render/` ever writes an escape; if an ANSI byte appears in plain-mode output it could only have come from upstream, and the `Stylize` identity-on-Off guarantee blocks that path.

Why a `Styler.Stylize(role, text)` decorator (Q1=A consequence) and not a renderer-level `r.WriteStyled(role, text)` method: - Adding methods to F18's `Renderer` interface forces every `Renderer` implementation (`ansiRenderer`, `plainRenderer`, future test fakes) to know about themes — couples plumbing to policy. - The decorator pattern lets plain mode statically prove “no ANSI” by giving the styler a `depth=Off` variant; the renderer never has to discriminate. - Tests pin byte sequences cleanly: `Stylize(theme.RoleError, "boom") == "\x1b[31mboom\x1b[0m"` is a one-line assertion against the production path, no renderer needed. - Migration is incremental: a call site that hasn't adopted the styler still emits unstyled text (no breakage); a call site that has adopted it gets coloured output in fancy mode and identical bytes in plain mode.

Why slash + env (Q5=B) and not cobra: - Theme is a *runtime preview* surface (a user wants to see what `light` looks like) — slash-command output is the natural fit (output is rendered through the active renderer, so `/theme show light` shows real ANSI when the renderer is fancy and clean text when plain). - A cobra subcommand `helixcode theme` would be a debug-only convenience; the CLI's `/theme slash` already covers the user-facing case. - Mid-process theme switching (re-running `theme.NewStyler` and re-threading it) is out of scope for v1; the slash command is read-only. F20.5 may add `/theme set <name>`.

3. Components

3.1 New files

- `helix_code/internal/theme/types.go` — Theme, Role enum, Color, ColorDepth, ThemeName, sentinel errors (ErrUnknownTheme, ErrInvalidYAML, ErrInvalidRole), constants.
- `helix_code/internal/theme/types_test.go`.
- `helix_code/internal/theme/builtin.go` — `BuiltIn()` `map[ThemeName]Theme`; concrete `ThemeDark / ThemeLight / ThemeNone` literals with full per-depth palettes.
- `helix_code/internal/theme/builtin_test.go`.
- `helix_code/internal/theme/detect.go` — `DetectThemeName(env func(string) string) (ThemeName, string)` (returns the resolved name + the *source* string for `/theme status`); `DetectColorDepth(env func(string) string) ColorDepth`. Pure functions of the env lookup; no `os.Getenv` inside the function body.
- `helix_code/internal/theme/detect_test.go` — table-driven over every (HELIXCODE_THEME, COLORFGBG, COLORTERM, TERM) combination called out in §3.6.
- `helix_code/internal/theme/loader.go` — `LoaderOptions (Env / ConfigDir / Filesystem seams)`, `Loader.Load() (*Styler, error)`, `Styler{Stylize}`.
- `helix_code/internal/theme/loader_test.go` — real temp `theme.yaml` with custom palette; injected `Env` and `ConfigDir`; assert merged Theme matches expected per-depth bytes.
- `helix_code/internal/commands/theme_command.go` — `ThemeCommand (Command impl)`; `/theme status`, `/theme list`, `/theme show <name>`.
- `helix_code/internal/commands/theme_command_test.go`.
- `helix_code/tests/integration/theme_test.go` — `//go:build integration`; ALWAYS-runs; real env vars via `LoaderOptions`. Env injection + real temp YAML; asserts byte-exact ANSI emission via the production `Styler`.
- `helix_code/tests/integration/cmd/p1f20_challenge/main.go` — runtime evidence harness.
- `challenges/p1-f20-theme-system/CHALLENGE.md + run.sh`.

3.2 Modified files

- `helix_code/cmd/cli/main.go` — three small additions: (1) construct the `theme.Loader + theme.Styler` at startup adjacent to the renderer; (2) wire the styler into the `handleGenerate non-stream RenderTextBlock` call (style response with `RoleInfo`, errors with `RoleError`); (3) register `/theme` via `commands.NewThemeCommand(c.styler, themeLoader)`.
- `helix_code/internal/commands/registry.go` — no schema change; one new registration via the existing `Register(cmd Command)` API.
- `helix_code/go.mod` — `gopkg.in/yaml.v3` is **already** an indirect dep (verify with `grep "yaml.v3" helix_code/go.mod` before T05); F20 promotes it to a direct dep at the version already in `go.sum`. T05 verifies `go mod tidy` produces only the indirect→direct line move; no new transitive entries.

One existing-tree direct dep promoted (yaml.v3); see §3.5.


```

        DepthANSI16                                // 16 colors via SGR
        30-37 / 90-97
    DepthANSI256                                    // 256 colors via
        "\x1b[38;5;<n>m"
    DepthTruecolor                                  // 24-bit RGB via
        "\x1b[38;2;<r>;<g>;<b>m"
)

// String returns "off"/"ansi16"/"ansi256"/"truecolor" for /theme
// status.
func (d ColorDepth) String() string

// ThemeName is a stable theme identifier. Built-ins are "dark",
// "light",
// "none". User YAML may declare any name; the loader treats
// names as opaque
// strings (case-sensitive, byte-for-byte equality).
type ThemeName string

const (
    ThemeDarkName ThemeName = "dark"
    ThemeLightName ThemeName = "light"
    ThemeNoneName ThemeName = "none"
)

// Color is a single styled-span code. The depth-specific Open
// string is the
// SGR sequence to emit BEFORE the styled text; Reset is shared
// across all
// depths and is the literal "\x1b[0m" sequence. An empty Open
// means "no
// styling at this depth" – equivalent to DepthOff for that role.
type Color struct {
    Open string // e.g. "\x1b[31m" (ANSI16), "\x1b[38;5;160m"
                // (ANSI256),
                // "\x1b[38;2;220;50;47m" (Truecolor), or ""
                (none)
}

// Reset is the universal SGR reset sequence. Exported as a const
// so tests can
// assert byte equality without re-typing the escape.
const Reset = "\x1b[0m"

// Theme is the per-depth palette for the 5 roles. The arrays are
// indexed by

```

```

// Role; ANSI16[RoleInfo] is the open code for Info at 16-color
    depth.
type Theme struct {
    Name      ThemeName
    ANSI16    [numRoles]Color
    ANSI256   [numRoles]Color
    Truecolor [numRoles]Color
}

// Sentinel errors. Tests compare via errors.Is.
var (
    ErrUnknownTheme = errors.New("theme: unknown theme name")
    ErrInvalidYAML   = errors.New("theme: invalid YAML override")
    ErrInvalidRole   = errors.New("theme: invalid role name")
)

// internal/theme/loader.go

type LoaderOptions struct {
    Env      func(string) string // default os.Getenv
    ConfigDir string          // default $XDG_CONFIG_HOME/
                helixcode (or $HOME/.config/helixcode)
    Filesystem fs.FS      // default os.DirFS — injected
                for tests
}

type Loader struct { /* unexported state */ }

func NewLoader(opts LoaderOptions) *Loader

// Load resolves the active theme name, depth, and any YAML
// override, then
// returns a constructed Styler. Returns wrapped ErrUnknownTheme
// if the
// resolved name does not match a built-in or YAML entry; wrapped
// ErrInvalidYAML if the YAML file exists but cannot be parsed.
func (l *Loader) Load() (*Styler, ResolvedSource, error)

// ResolvedSource records WHERE the active theme name came from.
// Used by
// /theme status output. One of "env" (HELIXCODE_THEME was set),
// "colorfgbg" (parsed from $COLORFGBG luminance), or "default"
// (fell
// through to "dark").
type ResolvedSource string

const (
    SourceEnv      ResolvedSource = "env"
    SourceColorFGBG ResolvedSource = "colorfgbg"

```



```

    SourceDefault    ResolvedSource = "default"
)

type Styler struct
    { /* unexported: theme + depth + name + source */ }

// Stylize wraps text with the role's open + reset SGR sequences.
//   - depth == Off           → returns text unchanged
//     (identity)
//   - depth has empty Open at role → returns text unchanged
//   - otherwise               → returns Open + text + Reset
//
// The function is allocation-free for the depth=Off path
//   (returns the input
// string directly) so the plain-mode hot path does not pay a
//   copy cost.
func (s *Styler) Stylize(role Role, text string) string

// Theme returns the active Theme value (read-only; do not
//   mutate).
func (s *Styler) Theme() Theme

// Depth returns the active ColorDepth.
func (s *Styler) Depth() ColorDepth

// Source returns the ResolvedSource for /theme status.
func (s *Styler) Source() ResolvedSource

```

3.4 Built-in palettes (concrete, load-bearing)

The exact open codes per role × depth for the three built-ins. Tests in T03 assert these byte-for-byte. Reset is "\x1b[0m" for every role.

dark **theme** — biased toward bright tones that read on dark terminals.

Role	ANSI16 (Open)	ANSI256 (Open)	Truecolor (Open)	RGB / SGR rationale
Info	\x1b[37m	\x1b[38;5;250m	\x1b[38;2;220;220;220m	bright grey / off-white (250) — neutral body text
Warn	\x1b[33m	\x1b[38;5;214m	\x1b[38;2;255;176;0m	amber (214) — pre-attentive but not alarming

Role	ANSI16 (Open)	ANSI256 (Open)	Truecolor (Open)	RGB / SGR rationale
Error	\x1b[31m	\x1b[38;5;196m	\x1b[38;2;255;64;64m	red (196) — high-saturation alarm
Highlight	\x1b[36m	\x1b[38;5;51m	\x1b[38;2;0;200;220m	cyan (51) — accent on dark background
Dim	\x1b[90m	\x1b[38;5;243m	\x1b[38;2;128;128;128m	bright-black / mid-grey (243) — supporting text

light theme — biased toward darker tones that read on light terminals.

Role	ANSI16 (Open)	ANSI256 (Open)	Truecolor (Open)
Info	\x1b[30m	\x1b[38;5;235m	\x1b[38;2;40;40;40m
Warn	\x1b[33m	\x1b[38;5;130m	\x1b[38;2;175;95;0m
Error	\x1b[31m	\x1b[38;5;124m	\x1b[38;2;175;0;0m
Highlight	\x1b[34m	\x1b[38;5;25m	\x1b[38;2;0;95;175m
Dim	\x1b[37m	\x1b[38;5;245m	\x1b[38;2;138;138;138m

none theme — empty Open for every role × depth. Stylize returns the input verbatim regardless of ColorDepth. Useful for users who want “TTY but no color” (e.g. piping through `less -R` is fine, but the user prefers monochrome).

3.5 New external dependencies

One indirect→direct promotion: `gopkg.in/yaml.v3` (already pulled in transitively by `viper/cobra`; verified before T05). No new entries in `go.sum`. T08’s verification step asserts `gomodtidy` produces no transitive additions.

Stdlib usage: `errors`, `fmt`, `io`, `io/fs`, `os`, `path/filepath`, `strings`, `strconv`. No CGO. Pure Go.

`golang.org/x/term` is unused in F20 — TTY detection is delegated to F18’s renderer factory (which is the source of truth for “is this writer a terminal”). The Styler’s `DepthOff` path is the structural enforcement of plain-mode-zero-color; the Styler does not need its own TTY probe.

3.6 Theme-name and depth resolution

Theme name (Q3=C) precedence ladder:

1. `HELIXCODE_THEME` env var. If set and non-empty:
 - If matches a built-in (`dark` / `light` / `none`) or a name in `YAML` → use it.
`Source = SourceEnv.`
 - If matches `NOTHING` → log a warning, fall through to step 2.
2. `$COLORFGBG` — terminal-emulator-supplied “;” indices (xterm convention; e.g. xterm sets `15;0` for white-on-black, gnome-terminal sets `7;0` for grey-on-black). Parse the *bg* index; if 0 (black) or in `{0,1,2,3,4,5,6,8}` the background is dark → choose `dark`; if in `{7,9,10,11,12,13,14,15}` → choose `light`. `Source = SourceColorFGBG.`
Unparseable values → fall through to step 3.
3. Default: `dark`. `Source = SourceDefault.`

Color depth (Q4=D) detection:

1. `$COLORTERM` value `truecolor` or `24bit` → `DepthTruecolor.`
2. Else `$TERM` matches glob `*-256color` (case-sensitive suffix check) → `DepthANSI256.`
3. Else `$TERM` is set, non-empty, not `dumb` → `DepthANSI16.`
4. Else (`$TERM` is `dumb` or unset/empty) → `DepthOff.`

When the renderer is in plain mode (F18 chose `plainRenderer`), the `Styler` is *additionally* forced to `DepthOff` regardless of what `DetectColorDepth` returned. This double-belts the no-leak invariant: even if a future regression causes the loader to emit `Truecolor` codes, plain mode short-circuits them at `Stylize` time.

3.7 YAML override schema

`$XDG_CONFIG_HOME/helixcode/theme.yaml` (Q2=C). File mode 0644 is acceptable (no secrets — `CONST-042` does not apply; see §9). Schema:

```
themes:
  custom-solarized:
    ansi16:
      info:      "\x1b[37m"
      warn:      "\x1b[33m"
      error:     "\x1b[31m"
      highlight: "\x1b[36m"
      dim:       "\x1b[90m"
    ansi256:
      info:      "\x1b[38;5;230m"
      warn:      "\x1b[38;5;136m"
      error:     "\x1b[38;5;160m"
      highlight: "\x1b[38;5;33m"
      dim:       "\x1b[38;5;240m"
    truecolor:
      info:      "\x1b[38;2;253;246;227m"
      warn:      "\x1b[38;2;181;137;0m"
      error:     "\x1b[38;2;220;50;47m"
      highlight: "\x1b[38;2;38;139;210m"
```

```

    dim:          "\x1b[38;2;101;123;131m"
partial-override:
ansi16:
    error: "\x1b[91m"    # only override Error; other roles
                        inherit from "dark"

```

Merge semantics: the loader resolves `HELIXCODE_THEME=<name>`. If `<name>` exists in YAML, the loader starts from the matching built-in (default `dark`; if the YAML entry sets a `base: light` field — F20.5 candidate — start from that built-in instead) and overlays any role × depth slot the YAML defines. Slots not mentioned in YAML keep the built-in code. **v1 merge is into the dark built-in** for all custom names; the optional `base:` field is deferred to F20.5.

Validation: - Unknown role keys (e.g. `infoo: ...`) → `ErrInvalidYAML` with the offending key. - Unknown depth keys (e.g. `ansi128: ...`) → `ErrInvalidYAML`. - Open string MUST start with `\x1b[` and end with `m` (the SGR delimiters). If it doesn't, `ErrInvalidYAML`. - File missing → no error; loader uses built-ins only. - File present but empty (zero bytes) → no error; built-ins only. - File present with parse error → wrap with `ErrInvalidYAML` and (per §5) `WARN` to `stderr` + fall back to built-ins (does NOT abort startup).

3.8 Existing-code constraints

- F18's `internal/render/` package is **unchanged**. Theme decorates content; the renderer emits bytes.
- F19's `internal/tools/askuser/` is **unchanged** for v1. `ask_user` prompts could be styled in F20.5 (e.g. `RoleHighlight` on the chosen choice's index in `/theme show-style preview`); v1 leaves them plain.
- The desktop GUI's `assets/colors/helix-theme.css` and `assets/colors/color-scheme.json` are **unchanged**. F20 targets the CLI exclusively; GUI theme synchronisation is OUT OF SCOPE (§8).

4. Data flow

4.1 Startup

```

cmd/cli/main.go::run():
├─ renderer, _ := render.NewRenderer(render.FactoryOptions{})
├─ defer renderer.Close()
├─ themeLoader := theme.NewLoader(theme.LoaderOptions{})
├─ styler, source, err := themeLoader.Load()
│   └─ if err != nil:
│       │   log warning + use built-in dark + DepthOff if YAML err, othe
├─ // If renderer is plain mode, force the styler to DepthOff so no ANSI
├─ if renderer.Mode() == render.ModePlain: styler = styler.WithDepth(the
├─ c.renderer = renderer
├─ c.styler    = styler
├─ // wire /theme slash command
    c.commandRegistry.Register(commands.NewThemeCommand(c.styler, themeLo

```

4.2 Per-call styling

```

caller (e.g. handleGenerate non-stream branch):
├─ resp := provider.Generate(ctx, req)

```

```

└ styled := c.styler.Stylize(theme.RoleInfo, resp.Text)
└ render.RenderTextBlock(c.renderer, "", styled)
  // styled in fancy mode = "\x1b[38;5;250m<resp.Text>\x1b[0m"
  // styled in plain mode = resp.Text (Stylize is identity due to De

```

4.3 /theme slash command

```

/theme                → alias of /theme status
/theme status         → prints: "active=<name> depth=<depth> source=<source>
                        three lines, each styled (heading via RoleHighlight
                        value via RoleInfo, footer via RoleDim).
/theme list           → prints all available theme names: built-ins + YAML.
                        One per line; the active one prefixed with "*" (Ro
/theme show <name>    → looks up the named theme; prints a 5-row table:
                        Role    Sample (styled at active depth)
                        info     "the quick brown fox"
                        warn     "...".
                        ...
                        Each sample is wrapped via Stylize using <name>'s p
                        (NOT the active styler) — so /theme show light on a
                        terminal previews the light palette codes.

```

The slash command’s output is itself rendered through the active renderer (F18) — so its bytes are subject to plain-mode-zero-color at the renderer boundary too. (Belt-and-suspenders: the previewed sample is wrapped in ANSI because the user explicitly asked to see what `light` looks like, and on a non-TTY destination the user would see those escapes as literal text — which is the *correct* behaviour for `show <name>` on a non-TTY: the user is asking “what bytes would this theme emit?”)

4.4 Resolution truth table

HELIXCODE_THEME	\$COLORFGBG	resolved name	\$COLORTERM	\$TERM	resolved
dark	*	dark	truecolor	xterm-256color	DepthT
light	*	light	24bit	xterm-256color	DepthT
none	*	none	*	*	(any dept codes are output is
nope	15;0	dark (fallback) + warn	truecolor	xterm-256color	DepthT
(unset)	15;0	dark (bg=0 → dark)	(unset)	xterm-256color	DepthA
(unset)	0;15	light (bg=15 → light)	(unset)	xterm	DepthA
(unset)	(unset)	dark (default)	(unset)	dumb	Depth0
(unset)	(unset)		(unset)	(unset)	Depth0

HELIXCODE_THEME	\$COLORFGBG	resolved name	\$COLORTERM	\$TERM	resolved
		dark (default)			
custom-solarized (in YAML)	*	custom-solarized	truecolor	xterm-256color	DepthT

4.5 Plain-mode short-circuit

When `renderer.Mode() == render.ModePlain`, the startup code overrides the Styler’s depth to `DepthOff` via a `WithDepth(DepthOff)` constructor (returns a new Styler — original is immutable):

```
if renderer.Mode() == render.ModePlain {
    styler = styler.WithDepth(theme.DepthOff)
}
```

`Stylize` at `DepthOff` returns the input text unchanged. A unit test asserts `Stylize(role, "x") == "x"` for every `role × DepthOff` combination, byte-for-byte. The Challenge harness’s `PLAIN-MODE-ZERO-COLOR` phase additionally constructs a real `plainRenderer` + a real Styler with dark theme + `DepthTruecolor` (then overridden to `Off`), writes 100 styled tokens through `RenderTextBlock`, and asserts the captured bytes contain `ZERO 0x1b` characters.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Unknown theme name in env** — `WARN` to `stderr` (`theme: HELIXCODE_THEME=<x>` not found; falling back to dark); proceed with default dark + `SourceDefault`. NEVER abort startup.
- **YAML parse error** — wrap as `fmt.Errorf("theme: %w: %s", ErrInvalidYAML, parseErr)`; `WARN` to `stderr`; proceed with built-ins only. NEVER abort startup.
- **YAML schema violation** (unknown role / depth / malformed Open code) — wrap as `ErrInvalidYAML` with the offending key; `WARN` to `stderr`; proceed with built-ins. The loader does NOT partially apply a malformed override (all-or-nothing per theme name).
- **Unknown role string in ParseRole** — return `ErrInvalidRole` with the unknown key. Used only by the YAML loader.
- **Unparseable \$COLORFGBG** — fall through to default; do NOT log (this signal is opportunistic, not configured).
- **\$COLORTERM unset + \$TERM unset** — `DepthOff`. Styler `Stylize` returns input unchanged. Equivalent to “no styling”.

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

Common bluff variants and their structural defences:

1. **(a) Theme “applied” but no ANSI bytes ever emitted** — the package compiles, `/theme list` shows three themes, but the call sites still pass raw `text` to the `renderer` (the

- migration in `cmd/cli/main.go:handleGenerate` was forgotten). **Defence:** the integration test wires a real `*os.File-backed bytes.Buffer` (via `FactoryOptions.IsTTY=true` injection) through the production `RenderTextBlock(c.renderer, "", c.styler.Stylize(theme.RoleInfo, body))` call site and asserts the captured bytes contain *both* `\x1b[` (an ANSI byte sequence — proves styling happened) AND the literal body text (proves the styling didn't replace the text). A Challenge phase BUILT-IN-DARK additionally counts `bytes.Count(captured, []byte("\x1b["))` and asserts it is exactly `2 * num-styled-tokens` (one open + one reset per token); a count of zero is a hard failure (bluff(a)).
2. **(b) Truecolor codes emitted on a 16-color terminal** — the depth detector or the loader picks Truecolor regardless of `$COLORTERM` value. **Defence:** a unit test sets `Env=fakeEnv({"TERM": "xterm"})` (no `COLORTERM`) and asserts `DetectColorDepth(env) == DepthANSI16`; another sets `Env=fakeEnv({"TERM": "xterm-256color"})` and asserts `DepthANSI256`; another sets `COLORTERM=truecolor + TERM=xterm-256color` and asserts `DepthTruecolor`. A Challenge phase DEPTH-AUTO-DETECT runs all four branches with synthesised env vars and asserts the resolved depth byte-for-byte against an expected sequence (`Off` for `TERM=dumb`, etc.).
 3. **(c) Plain mode leaks ANSI through the theme path** — a regression in the Styler emits ANSI even when `DepthOff`, OR a call site bypasses the Styler and calls the package-level `BuiltIn()["dark"].Truecolor[RoleInfo]` directly. **Defence:** a unit test constructs a `Styler{depth: DepthOff, theme: ThemeDark}` and asserts `Stylize(RoleInfo, "x") == "x"` byte-for-byte. The Challenge's PLAIN-MODE-ZERO-COLOR phase wires a real `plainRenderer` + real Styler with dark theme through `RenderTextBlock` and asserts `bytes.IndexByte(captured, 0x1b) == -1`. A second source-scan unit test greps `cmd/cli/` and `internal/commands/theme_command.go` for any direct reference to `theme.ThemeDark.Truecolor[/theme.ThemeDark.ANSI256[/theme.ThemeDark.ANSI16[` (i.e. bypassing the Styler) — must return zero matches.
 4. **(d) Theme env var honoured but COLORFGBG/COLORTERM detection branches never triggered in tests** — a regression in the env detector silently downgrades every user (e.g. always returns `DepthOff`). **Defence:** the table-driven detect tests in T04 enumerate every branch of the §3.6 ladder with a synthetic `Env` closure (no `os.Setenv` — pure functions). The Challenge's DEPTH-AUTO-DETECT phase uses synthesised env vars (a `func(string) string` closure passed via `LoaderOptions.Env`) and asserts the resolved name AND depth for each of 6 representative env combinations.

Required real-execution criteria (define what “theme system works”):

1. **Unit tests** assert exact ANSI byte sequences for each (role × depth) combination of each built-in theme. T03's tests pin §3.4's table byte-for-byte (e.g. `dark.Truecolor[RoleError].Open == "\x1b[38;2;255;64;64m"`).
2. **Plain-mode invariant** — `Stylize(role, text) == text` byte-identical when `Depth == Off`, for every role and every theme. Captured plain-renderer output contains zero `0x1b` bytes regardless of theme. Source-scan ensures no direct package-level palette access bypasses the Styler.
3. **Color-depth auto-detect** is tested with an injected `Env func(string) string` closure; covers all four branches (`Truecolor`, `ANSI256`, `ANSI16`, `Off`) in §3.6 + the dumb and unset-TERM cases.
4. **Challenge harness** — five always-run phases (BUILT-IN-DARK + BUILT-IN-LIGHT + PLAIN-MODE-ZERO-COLOR + DEPTH-AUTO-DETECT + YAML-OVERRIDE), each with positive byte evidence.

Concrete forbidden-phrases anti-bluff smoke:

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/theme internal/commands/theme_command.go && echo
BLUFF || echo clean
```

Must always print clean.

5.3 Concurrency

Styler is read-only after construction. Stylize is a pure function of (theme, depth, role, text); concurrent reads are safe. The loader is called once at startup; no runtime mutation. Future runtime theme switching (F20.5 / theme set) would require an atomic.Value swap or a copy-on-write Styler — out of scope for v1.

5.4 No-color terminals (NO_COLOR=1)

Out of scope for v1. The convention <https://no-color.org/> (treat NO_COLOR as “disable color regardless”) is a F20.5 candidate. v1 documents this gap explicitly: a user with NO_COLOR=1 set will still see ANSI in fancy mode unless they additionally set HELIXCODE_THEME=none. F20.5 will add a fourth precedence rung that maps NO_COLOR non-empty → theme=none.

5.5 Cross-depth fallback within one theme

A theme with empty Open at depth Truecolor (e.g. user-supplied YAML that only defined ANSI16 codes) does NOT fall back to ANSI16 at runtime. The Styler returns text unchanged for that role × depth combination. Rationale: silently substituting a different depth’s code can warp the palette (a custom theme designed only for 16-color may use \x1b[31m for both Error and Warn at ANSI16; falling back to ANSI16 codes when the terminal supports Truecolor would *correctly* render those colors, but loses the designer’s chance to differentiate at Truecolor depth). v1 keeps the rule simple: empty Open → no styling for that slot. F20.5 may add an opt-in inherit: true field per slot.

6. Testing

6.1 Unit (real bytes . Buffer capture; injected env closure; no mocks of stdlib)

Types (types_test.go): - TestRole_String_LowercaseNames. -
TestRole_ParseRole_OK_ForAll15. - TestRole_ParseRole_Unknown_Err. -
TestColorDepth_String_AllFourValues. -
TestThemeName_StableConstants. - TestColor_OpenZeroValueIsEmpty. -
TestReset_LiteralByteSequence. -
TestErrorSentinels_DistinctErrorsIs.

Built-ins (builtin_test.go): -
TestBuiltIn_DarkAllRolesAllDepths_BytesPinned — assert each row of §3.4’s dark table byte-for-byte. - TestBuiltIn_LightAllRolesAllDepths_BytesPinned. -
TestBuiltIn_NoneAllOpenAreEmpty_AllDepthsAllRoles. -

TestBuiltIn_RegistryReturnsThreeThemes.-
TestBuiltIn_NamesMatchConstants.

Detection (detect_test.go): - TestDetectThemeName_EnvSet_ReturnsEnv —
Env={"HELIXCODE_THEME": "light"} → ("light", "env").-
TestDetectThemeName_EnvUnknown_FallsThroughToColorFGBG —
Env={"HELIXCODE_THEME": "nope", "COLORFGBG": "15;0"} → ("dark",
"colorfgbg").- TestDetectThemeName_NoEnv_ColorFGBGDark_ReturnsDark
— Env={"COLORFGBG": "15;0"} → ("dark", "colorfgbg").-
TestDetectThemeName_NoEnv_ColorFGBGLight_ReturnsLight —
Env={"COLORFGBG": "0;15"} → ("light", "colorfgbg").-
TestDetectThemeName_NoEnv_NoColorFGBG_ReturnsDarkDefault — Env={} → ("dark", "default").-
TestDetectColorDepth_ColorTermTruecolor_Truecolor.-
TestDetectColorDepth_ColorTerm24bit_Truecolor.-
TestDetectColorDepth_TermXterm256_ANSI256.-
TestDetectColorDepth_TermXterm_ANSI16.-
TestDetectColorDepth_TermDumb_Off.-
TestDetectColorDepth_TermUnset_Off.

Loader + Styler (loader_test.go): - TestLoader_NoYAMLFile_BuiltInsOnly.-
TestLoader_YAMLFileMissing_NoError.- TestLoader_YAMLEmpty_NoError.-
TestLoader_YAMLParseErr_WrappedErrInvalidYAML.-
TestLoader_YAMLSchema_UnknownRole_ErrInvalidYAML.-
TestLoader_YAMLSchema_UnknownDepth_ErrInvalidYAML.-
TestLoader_YAMLSchema_OpenMissingPrefix_ErrInvalidYAML.-
TestLoader_YAMLOverridesBuiltInDark_PerSlot.-
TestLoader_YAMLOverridesBuiltIn_UnmentionedSlotsKeepBuiltIn.-
TestStyler_Stylize_DepthOff_IdentityForAllRoles.-
TestStyler_Stylize_DepthANSI16_DarkInfo_BytesPinned.-
TestStyler_Stylize_DepthANSI256_DarkError_BytesPinned.-
TestStyler_Stylize_DepthTruecolor_DarkHighlight_BytesPinned.-
TestStyler_Stylize_NoneAllDepths_AllRolesIdentity.-
TestStyler_Stylize_EmptyOpen_NoStyling — covers cross-depth gap per §5.5. -
TestStyler_WithDepth_ReturnsNewStylerCorrectDepth — original immutable.
- TestStyler_Source_Theme_Depth_Accessors.-
TestNoDirectPaletteAccess_BypassesStyler_SourceScan — greps cmd/
cli//internal/commands/theme_command.go for theme.Theme*\
(Truecolor|ANSI256|ANSI16) \[; FAILs on any hit (anti-bluff (c)).

Theme command (theme_command_test.go): -
TestThemeCommand_Name_IsTheme.-
TestThemeCommand_Status_PrintsActiveNameAndDepthAndSource.-
TestThemeCommand_List_IncludesAllBuiltIns.-
TestThemeCommand_List_IncludesYAMLOverrides.-
TestThemeCommand_Show_KnownName_RendersFiveRoles.-
TestThemeCommand_Show_UnknownName_ErrUnknownTheme.-
TestThemeCommand_Show_UsesNamedThemePalette_NotActive — /theme
show light on a dark styler shows light's codes.

6.2 Integration (//go:build integration)

tests/integration/theme_test.go (ALWAYS-runs; no infrastructure dep):

- TestTheme_Integration_FancyRenderer_DarkInfoEmitsANSI — wires real ansiRenderer (via FactoryOptions{IsTTY: ()=>true}) + real Styler (dark + DepthTruecolor) through render.RenderTextBlock; assert captured bytes contain \x1b[38;2;220;220;220m AND the literal text AND \x1b[0m.
- TestTheme_Integration_PlainRenderer_DarkInfoEmitsZeroAnsi — wires real plainRenderer (via FactoryOptions{IsTTY: ()=>false}) + real Styler force-overridden to DepthOff; assert bytes.IndexByte(captured, 0x1b) == -1 AND captured contains the literal text.
- TestTheme_Integration_DepthAutoDetect_AllFourBranches — runs through Loader.Load() four times with synthesised Env covering the four depth branches; asserts each Styler.Depth() matches expected.
- TestTheme_Integration_YAMLOverride_CustomNameLoads — writes theme.yaml to a tempdir; sets LoaderOptions.ConfigDir + Env={"HELIXCODE_THEME": "custom-solarized"}; loads; asserts Stylize(RoleError, "x") returns the YAML-supplied open code + reset.

6.3 Challenge (challenges/p1-f20-theme-system/)

Five-phase output skeleton (all always-run):

```
=== THEME-PHASE-A: BUILT-IN-DARK (always runs) ===
[PASS] constructed real ansiRenderer + Styler(dark, Truecolor)
[PASS] RenderTextBlock(renderer, "", styler.Stylize(RoleInfo, "hello world")
[PASS] captured bytes contain "\x1b[38;2;220;220;220m" (dark Info Truecolo
[PASS] captured bytes contain literal "hello world"
[PASS] captured bytes contain "\x1b[0m" (reset)
[PASS] bytes.Count(captured, "\x1b[") == 2 (exactly one open + one reset)

=== THEME-PHASE-B: BUILT-IN-LIGHT (always runs) ===
[PASS] constructed real ansiRenderer + Styler(light, Truecolor)
[PASS] Stylize(RoleError, "boom") wrapped with "\x1b[38;2;175;0;0m" + "\x1
[PASS] captured bytes contain expected open + literal + reset in correct o
[PASS] byte offset of open < byte offset of literal < byte offset of reset

=== THEME-PHASE-C: PLAIN-MODE-ZERO-COLOR (always runs) ===
[PASS] constructed real plainRenderer + Styler(dark, depth-overridden to 0
[PASS] wrote 100 styled tokens through RenderTextBlock
[PASS] bytes.IndexByte(captured, 0x1b) == -1 (zero ANSI bytes)
[PASS] captured bytes contain all 100 token literals

=== THEME-PHASE-D: DEPTH-AUTO-DETECT (always runs) ===
[PASS] env={COLORTERM:truecolor, TERM:xterm-256color} → DepthTruecolor
[PASS] env={TERM:xterm-256color} → DepthANSI256
[PASS] env={TERM:xterm} → DepthANSI16
[PASS] env={TERM:dumb} → DepthOff
[PASS] env={} (no TERM, no COLORTERM) → DepthOff
[PASS] env={HELIXCODE_THEME:nope, COLORFGBG:15;0} → name="dark" source="co

=== THEME-PHASE-E: YAML-OVERRIDE (always runs) ===
```

```
[PASS] wrote theme.yaml to tempdir with custom-solarized palette
[PASS] LoaderOptions.ConfigDir=<tempdir>; Env={HELIXCODE_THEME:custom-sola
[PASS] Loader.Load() returned no error; Styler.Theme().Name == "custom-sol
[PASS] Stylize(RoleError, "x") at DepthTruecolor wraps with "\x1b[38;2;220
[PASS] Slot not in YAML (RoleHighlight at ANSI16) inherits from dark "\x1b
```

SUMMARY: PHASE-A=6/6 PASS; PHASE-B=4/4 PASS; PHASE-C=4/4 PASS; PHASE-D=6/6

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable. Byte-count assertions (Phase A: exactly 2 ANSI sequences per token) and zero-byte invariants (Phase C: zero `0x1b` bytes in plain mode) are positive evidence of real ANSI emission and real plain-mode purity.

7. Cross-platform

ANSI escape codes are emitted on Linux, macOS, and Windows. Windows 10+ has VT-mode auto-enabled by Go's runtime in `F18(runtime · syscall_windows.go); cmd / PowerShell` consoles render `\x1b[31m` correctly. Pre-Windows 10 is OUT OF SCOPE (matches F18's stance).

`$COLORTERM` and `$TERM` env vars are set by terminal emulators on all three platforms (xterm, gnome-terminal, iTerm2, Windows Terminal, ConEmu). The detection algorithm in §3.6 is identical across platforms (no platform branches in `detect.go`).

The cross-compile `make prod` target (linux/macos/windows) is exercised in T08. No platform-specific code in F20.

8. Out of scope (deferred)

- **Bold / italic / underline / strikethrough / blink / reverse-video** — F20.5. The YAML schema gains optional sibling fields (`bold: true, italic: true, ...`) per role × depth slot.
- **Background colors** — F20.5. The YAML schema adds an optional `bg` field whose Open uses SGR `48; . . .` codes.
- **Per-tool-type custom colors** (e.g. all `lsp_diagnostic` results in red regardless of role) — F20.5; would need a tool-id → role-override map.
- **Gradient text** (per-character interpolated colors) — out of scope indefinitely (high complexity, low ROI).
- **NO_COLOR env var** support per <https://no-color.org/> — F20.5; new precedence rung (highest priority) mapping `NO_COLOR` non-empty → `theme=none`.
- **GUI theme synchronisation** with the desktop Fyne UI — out of scope. The CSS / JSON theme assets are a separate pathway.
- **Mid-process theme reload** on YAML file change — F20.5; would need fsnotify watcher + atomic swap.
- `/theme set <name>` runtime switching — F20.5; would need atomic-value Styler swap.
- **Cross-depth fallback within one theme** (e.g. ANSI16-only theme rendering on Truecolor terminal) — F20.5; would need an opt-in `inherit: true` slot field.
- **Cobra subcommand** `helixcode theme` — debug-only convenience; F20.5 candidate.
- **256-color truecolor approximation** — out of scope. A Truecolor-only theme on an ANSI256 terminal degrades to no-styling for those slots (per §5.5).

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has FIVE always-run phases. Every phase records positive runtime evidence: byte counts (Phase A: exactly 2 ANSI sequences per token), byte content (Phase B: open/literal/reset present in order), zero-byte invariants (Phase C: zero `0x1b` in plain-mode capture), depth-resolution byte-equality (Phase D: synthesised `env` → expected depth), YAML-merge byte-equality (Phase E: custom slot wraps with custom code; un-mentioned slot wraps with built-in code). Every byte-evidence mismatch is a hard failure.
- **CONST-039** — Challenge at `challenges/p1-f20-theme-system/` + evidence harness at `tests/integration/cmd/p1f20_challenge/main.go`. Every phase asserts positive byte evidence.
- **CONST-042 (No-Secret-Leak)** — theme YAML carries NO secrets (color codes are non-sensitive). File mode **0644** is acceptable (no secrets means no `.env` analogue; the file lives in `$XDG_CONFIG_HOME/helixcode/theme.yaml` which is the user's own config dir). The loader does NOT log file contents; only the resolved theme name is logged at INFO level. A unit test scans `internal/theme/*.go` for any `logger.Info(.*\ (palette\|color\|open\|reset\))` match and FAILs on any hit (defensive — there are no actual secrets, but log discipline is consistent with F19's anti-bluff posture).
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force; explicit user authorization is requested at T09 before pushing.
- **No-Mocks-In-Production (Universal Rule 2)** — the loader's only test seams are constructor-injected (`Env func(string) string, ConfigDir string, Filesystem fs.FS`); no filesystem abstraction in production paths. The Styler has zero mockable surface — it's a pure value type. Unit tests use `real bytes.Buffer` for I/O capture and `real os.MkdirTemp + os.WriteFile` for YAML files. Mock renderers do NOT appear; the integration test wires the `real ansiRenderer` and the `real plainRenderer` from F18.
- **CONST-033 (No host power management)** — F20 emits ANSI escape codes only; never `\x1b[5n` device-status-report or any escape with side effects on the host. The emitted SGR sequences (`\x1b[31m`, `\x1b[38;5;<n>m`, `\x1b[38;2;<r>;<g>;<b>m`, `\x1b[0m`) modify only terminal display state.

10. Open questions resolved

Q	Answer	Resolution
Q1: role palette	(A) 5 semantic roles: info / warn / error / highlight / dim	Consistent role-based palette applied across LLM output, tool results, slash output
Q2: defaults + override	(C) Built-in dark/light/none + optional user YAML override	Built-ins ship with binary; YAML at <code>\$XDG_CONFIG_HOME/helixcode/theme.yaml</code> overlays per role × depth slot
Q3: name precedence	(C) <code>HELIXCODE_THEME</code> > <code>\$COLORFGBG</code> parse > default dark	Three-rung ladder; unparseable signals fall through to default
Q4: depth detection	(D) <code>\$COLORTERM</code> truecolor/24bit → Truecolor; <code>\$TERM</code>	Source-of-truth for color depth at startup; renderer

Q	Answer	Resolution
	* - 256color → ANSI256; set-but-not-256 → ANSI16; dumb/unset → Off	plain-mode further forces depth=Off
Q5: surface	(B) Env var (HELIXCODE_THEME) + / theme slash (status / list / show); NO cobra	Read-only slash; runtime switching is F20.5

11. Non-obvious decisions (recorded for plan-time review)

- Decorator pattern (Stylize wraps text) vs renderer extension** (`r.WriteStylized(role, text)`) — picked decorator. Keeps F18's `Renderer` interface unchanged; plain mode statically proves no-leak via `depth=Off`; tests pin byte sequences without instantiating a renderer; migration is incremental (un-migrated call sites still emit unstyled text correctly). Recorded in §2 trailer.
- Five roles, not three or seven** — `Info / Warn / Error / Highlight / Dim` covers the 80% case (status output, warnings, errors, accents, supporting text). Three is too few (no accent); seven adds noise (`Success` overlaps `Info`, `Critical` overlaps `Error`). Recorded in §1.
- Depth as a per-Theme array ([5]Color × 3 depth tiers) NOT a runtime cross-depth conversion** — a theme designer hand-picks codes per depth, so a `Truecolor` palette is not auto-derived from `ANSI16` (which would lose information) or auto-quantised from `RGB` (which would warp warm/cool tones). The cost is more `YAML` lines for a custom theme; the benefit is faithful palette rendering at every depth. Recorded in §3.3.
- \$COLORFGBG parsing is opportunistic, NOT configured** — a malformed value falls through silently (no warn). Rationale: terminal emulators set this inconsistently (`xterm` sets `15 ; 0`, `gnome-terminal` sets `7 ; 0`, `alacritty` does not set it at all); warning on unparseable values would `WARN`-spam every `alacritty` user. Recorded in §5.1.
- Plain mode forces depth=Off via WithDepth(DepthOff) at startup** — belt-and-suspenders against future regressions. Even if `DetectColorDepth` returns `Truecolor`, plain-mode `Stylize` is identity. The test that pins this is `TestStyler_Stylize_DepthOff_IdentityForAllRoles`. Recorded in §4.5.
- YAML merge always starts from dark in v1** — no `base:` field. Rationale: merge semantics are simple (overlay slot-by-slot); the optional base-selection is F20.5. `v1` does the limitation in §3.7.
- none theme is distinct from depth=Off** — `none` means “I’m on a TTY but I want no color” (e.g. piping through `less -R` is fine but the user prefers monochrome); `depth=Off` means “the terminal can’t render ANSI”. Both produce identical bytes (`Stylize(role, x) == x`); the distinction matters for `/theme status` (source / depth reporting) and for the test that pins each path independently.
- The Styler does NOT probe TTY itself** — F18's renderer factory is the source of truth. F20's startup code reads `renderer.Mode()` and conditionally overrides the `Styler`. Rationale: two probes would risk drift; the renderer is the authority on output destination.
- /theme show <name> previews the named theme's palette, NOT the active styler's** — so `/theme show light` on a dark-themed session shows `light`'s codes (not `light`'s codes mapped through `dark`'s depth). Rationale: the user is asking “what does `light` look like”; the answer must use `light`'s data. Recorded in §4.3.
- `bytes.Count(captured, []byte("\x1b[")) == 2 * num-tokens` as the Phase A invariant — exactly one open + one reset per styled token. A count of zero proves

bluff (a) (no styling); a count off by N proves bluff (c) (extra ANSI leaks). Recorded in §5.2.

11. **The theme.yaml file is read once at startup** — no fsnotify watcher in v1. F20.5 may add hot-reload; v1 documents this in §8 and §11. Rationale: the theme is a startup-property like the renderer mode (F18 is also startup-only); both are configured before the agent loop opens.
12. **yaml.v3 indirect→direct promotion is the only go.mod change** — verified before T05 by `grep "yaml.v3" helix_code/go.mod`. T08 verifies `go mod tidy` produces only the `require` block move; no new `go.sum` entries.
13. **CSS / JSON assets in assets/colors/ are unchanged** — those drive the Fyne desktop GUI; coupling them to the CLI theme would require a runtime CSS parser in Go (out of scope). Recorded in §3.8 + §8.
14. **Phase 1 close-out** — F20 is the final Phase 1 feature. T09 records the programme milestone in PROGRESS.md alongside the F20 close-out summary.